Tic-Tac-Toe as a Zero-Knowledge Proof Problem

Ashfi H.M. Shah (Queen's University – School of Computing, 25rrym@queensu.ca)
Nazmul Islam (Queen's University – School of Computing, nazmul.i@queensu.ca)
Abdelaziz Al-Husni (Queen's University – School of Computing, a.alhusni@queensu.ca)
Dr. Furkan Alaca (Queen's University – School of Computing, furkan.alaca@queensu.ca)

ABSTRACT

This research seeks to analyse the game of Tic-Tac-Toe as a Zero-Knowledge Proof problem. Tic-Tac-Toe is a deterministic game which requires a player to use a strategy to win. A win can be accomplished in various ways - and each state that can be classified as a win can be brought in various ways - at least 12 configuration of sequential states to achieve such. But knowledge of which way was used should not be relevant in judging whether or not a player has won. Regardless of whether or not a player won a game, they can lie that they did. But we can see if they actually won without actually seeing the intermediary steps using zk-SNARKs. Techniques one can use are hashing and time-stamps to certify results and confirm no incidents of cheating or forgery. This research will use Python to implement a zk-SNARKs based algorithm specialised towards certifying Tic-Tac-Toe results.

1 INTRODUCTION

Tic-Tac-Toe is a two-player deterministic game where one player plays as 'x', and the other player plays as 'o'. There is a 3x3 grid and each player ideally wants to make a line of three – whether that be horizontal, vertical, or diagonal. Figure 1 shows a sample of possible game configurations that can result in the green player 'X' winning or there being a stalemate. To win this game – similar to other one-versus-one games like chess or checkers – the player should uphold secrecy of moves they are going to make. Players should be able to prove they have won a game without revealing moves they have made or were thinking about. This introduces a zero-knowledge proof problem.

Zero-knowledge proofs are a cryptographic technique that helps provers prove to verifiers that a certain scenario is true. However, to achieve said task, the prover cannot introduce any other information except for the point that he or she wishes to prove. The effectiveness of the proof must be absolute – either complete or non-existent. [6]

Suppose that we have Alice playing Tic-Tac-Toe. Alice has a line of three and wants to prove that she won the game to Bob, who was not watching the game. A zero-knowledge proof would ensure that she would prove she won the game without going into detail of intermediary steps (Figure 2) she took. This can be done as simply as showing that there is a line of three. There is a board with a line of three, and maybe some other 'x's and 'o's but we do not care in which order they were placed – and for that reason Bob does not need to know either.

We propose to demonstrate this by developing a Python program. It will start with a simple Tic-Tac-Toe program – but ensure the program does not remember the previous steps. Only the present board. Each present board will evaluate whether or not anyone has won. And when someone has won, it will reveal which three cells imply such result.

Research Problem: Can we prove that a player Alice has won a game of Tic-Tac-Toe without analysing what moves she and her competitor played?

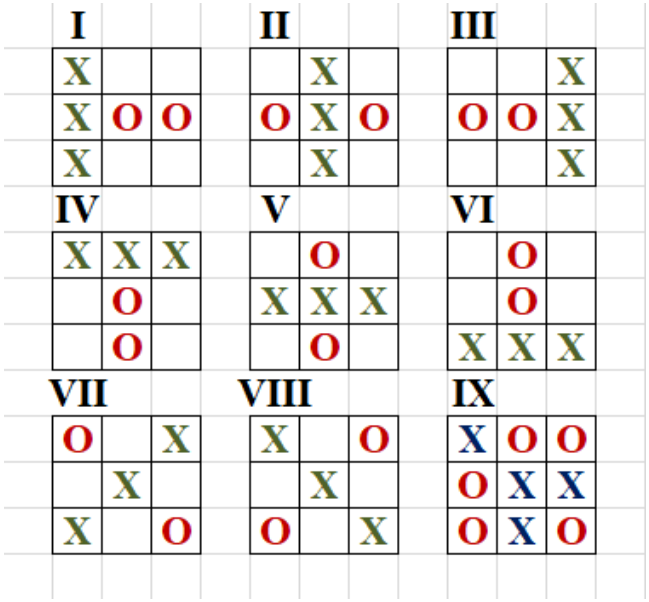

Figure 1: A Sample of Outcomes of Tic-Tac-Toe, Including all Winning Strategies for 'X' (I-VIII), and a Stalemate Option (IX)

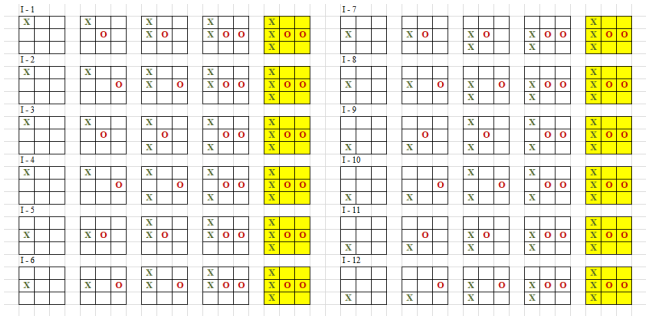

Figure 2: All Possible Intermediary Steps to Achieve Scenario I (Yellow) from Figure 1

2 ZERO-KNOWLEDGE PROOFS BACKGROUND

There is precedent of Tic-Tac-Toe being implemented as a zero-knowledge proof. Wong created a Tic-Tac-Toe using the Succinct Zero-Knowledge Non-Interactive Arguments of Knowledge (zk-SNARKs) scheme. [9] zk-SNARKs uses three arithmetic algorithms and is very scalable for Tic-Tac-Toe. My program will take advantage of this and develop these algorithms.

Wong had used libraries such as snarkyjs and the Mina Blockchain. [9] `Snarkyjs` is a library that introduces Javascript and is compatible with the Mina Blockchain. It introduces useful datatypes and functions. [9] These are some possible options that can be considered if necessary but we will take a more conservative approach and minimise the number of libraries used for the development.

2.1 What are zk-SNARKs?

zk-SNARKs is an update from the SNARK algorithm which adds the property of zero-knowledge [5], making it viable for this project. zk-SNARKs is an acronym of the following features [5]:

Zero Knowledge (zk-): Enables the SNARKs proof to be done without revealing intermediary steps.

Succinct (S): The proof is smaller than the witness (intermediary steps) and computations of such.

Non-interactive (N): Does not need multiple interactions to prove, but rather proves in one step of communication.

Argument (AR): Provers should have enough computational resources to convince the verifier.

Knowledge-sound (K/Ks): The prover (Alice) should know the witness - the intermediary steps. However, the verifier (Bob) would not know this given the problem is zero-knowledge.

According to Consensys, this scheme to use three algorithms – a key generator, prover, and verifier. [2] Nitulescu mentioned that "Zero-Knowledge Succinct Non-interactive Arguments of Knowledge (zk-SNARKs) are non-interactive systems with short proofs (i.e., independent of the size of the witness) that enable verifying NP computations with substantially lower complexity than that required for classical NP verification." [5] She identified three steps[5] that play a role in this process (Figure 3):

(1) **Artithmetisation** - translation of computations or circuits into arithmetic relations
(2) **Building a proof system** - Using these arithmetic relations to design a information-theoretic proof system
(3) **Compiling the proof system** - Using cryptographic tools, the proof system can be compiled into an efficient system.

zk-SNARKs Proofs can be either QAP-based (Quadratic Arithmetic Program) or PIOP-based. [5] Rahimi et al. discussed that Quadratic Arithmetic Program based zk-SNARKs is an efficient form of zk-SNARKs. They defined QAP as representation of an arithmetic circuit using polynomials, such that the correctness of the computation can be checked using a single polynomial condition. [7] In addition, QAP-based zk-SNARKs uses the three algorithms prescribed by Consensys. [2] Nitulescu also mentioned that there is a second type of zk-SNARKs, which is Polynomial Interactive Oracle Proofs (PIOP) [5]. This is "an interactive proof system in which the prover computes polynomials and the verifier can query these polynomials at evaluation points of her choice". [10][1].

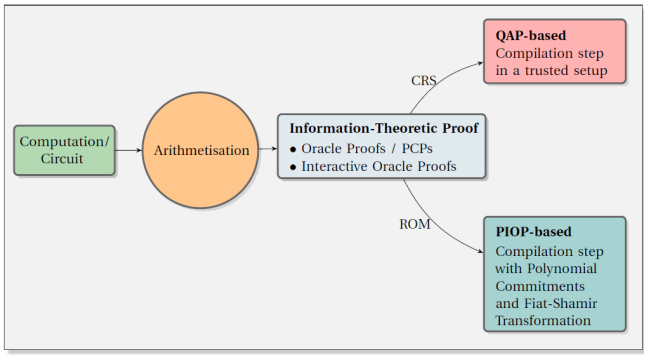

Figure 3: Steps of zk-SNARKs Outlined by Nitulescu [5]

Given our problem, it is more efficient to use QAP as the type of zk-SNARK.

2.2 The Three Algorithms of zk-SNARKs

As discussed, zk-SNARKs tends to have three algorithm for achieving necessary tasks expected from said system.

The first algorithm is the setup or key generator algorithm. This is usually done by a *trusted party*. [7] It "takes a secret parameter `lambda` and a program `C`, and generates two publicly available keys, a proving key `pk`, and a verification key `vk`. These keys are public parameters that only need to be generated once for a given program `C`" [2].

The second algorithm is called the prover algorithm. It generates a proof that "the prover knows a witness `w` and that the witness satisfies the program". Doing so comes in two parts - first "the prover builds the arithmetic circuit and calculates the values of all wires" in the circuit that corresponds to our computations [7] followed by running functions that compute the proof. [7].

The third algorithm is the verifier algorithm, which computes whether or not the proof is valid. It "returns true if the prover knows a witness `w` satisfying `C(x,w) == true`" and false otherwise. [2]

2.3 What is Mina Blockchain?

The *Mina Protocol*, first developed in 2017, is an "uniquely small blockchain that utilizes zero-knowledge smart contracts (zkApps) to offer a private, secure, and scalable infrastructure for the web3 ecosystem" which uses decentralisation "to ensure a robust network". [3] It has proven applicability to zero-knowledge problems including cryptocurrency, banking sector, and certifying election results. [3] In addition, it has proven it can be applied to this problem, as it has been used by Wong et al. [9]

"Mina Protocol uses a set of hybrid Zero-Knowledge proofs (zk-SNARKs), a unique node structure, and a technical framework. At the most basic level, Zero-Knowledge proofs are a technology that allows users to prove they have certain information or are a particular person without revealing any of the actual data to the other party. Building on this, the Mina Protocol introduces a mechanism called Recursive Zero-Knowledge Proof." [3]

3 METHODOLOGY

In this implementation, we first need to identify features of a QAP-inspired zk-SNARKs algorithm. Table 1 does that for us.

Table 1: Mapping of QAP/zk-SNARK Concepts in our System

QAP / zk-SNARK	Your System
Circuit	Tic-Tac-Toe rules
Witness (w)	Game logs
Statement	"X wins"
Prover	Commitments + Validation
Verifier	Verification function

To conduct this experiment, a program was developed using Python on Google Colab and is available on GitHub at https://github.com/Ashfi-Hasnain/Tic-Tac-Toe-as-a-Zero-Knowledge-Proof-Problem.git. [8]

The program is simpler compared to that used by Wong, [9] as it did not use specialised libraries like `Snarkyjs`. Rather, this implementation used three simple Python libraries like `hashlib`, `time`, and `random` for the respective tasks of hashing, time stamping, and randomisation. [8]

The first section of the code is the game engine. It defines a class that facilitates necessary functions of the Tic-Tac-Toe game. The game includes functions for initialisation, displaying board, executing moves, switching current players (to ensure each player gets only one turn at a time), checking wins, ending games that result in draws, getting available moves, and randomising moves. It should be mentioned that both Alice (X) and her rival (O) place their moves through pure randomisation and not through strategic means. The implications will be discussed later on.

There is also a function that is designed to automate the game-play, playing as both Alice (X) and her rival (O). This and the previous section of the code plays the setup role for our zk-SNARKs implementation.

Given that this is a zero-knowledge proof problem, we want to ensure Alice's claim is genuinely true without actually verifying step by step. The intermediate steps are secret yet vital to prove her claim. How can this be done? What we do is use hashes to conceal the intermediate steps and add timestamps to ensure these "intermediate steps" are not forged. These secured information are created using SHA-256 hashing and are provided to the verifier.

The prover algorithm is partly involved in the game-play as well, as it involves Alice's claim that she won. Whether or not she's telling the truth, she does have access to a witness of the information - her private log. However, the verifier (Bob), will noth have access to said private logs, but rather a public log. So he will use this public log instead to verify if she won or not. This is done in the verifier function.

All of these functions and final displays are commanded by the main function. Additionally, said main functions filter out games with a result of draws.

3.1 Phases of Development

The program was developed in four phases as described:

(1) **Development of a Tic-Tac-Toe game**
(2) **Automation of the game (randomisation)** - See Figure 4 for its log.
(3) Implementation of 'zk-SNARKs' without verifier (*What would become verifier had knowledge of classified information and thus would not be compliant to zk-SNARKs*)
(4) **A complete zk-SNARKs system** (*but game engine did not generate cases of Alice losing*)
(5) **Introduced cases of Alice's claim being false**

3.2 Game Logs and their Information

There are two versions of the game logs. A private version is retained by Alice as her witness. However, if Bob were to have access to this log, this would violate the concept of zero-knowledge. That is why, we also need to develop a public version - which is then used by the verifier.

Early on, a primitive log interface was developed to display what occurred. Figure 4 shows what it would have looked like. Eventually, this information, while retained in the private version, was either removed or hashed for secrecy in the public version.

```
Game Log:
{'player': 'X', 'move': 8, 'board': [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', 'X']}
{'player': 'O', 'move': 7, 'board': [' ', ' ', ' ', ' ', ' ', ' ', ' ', 'O', 'X']}
{'player': 'X', 'move': 0, 'board': ['X', ' ', ' ', ' ', ' ', ' ', ' ', 'O', 'X']}
{'player': 'O', 'move': 3, 'board': ['X', ' ', ' ', 'O', ' ', ' ', ' ', 'O', 'X']}
{'player': 'X', 'move': 1, 'board': ['X', 'X', ' ', 'O', ' ', ' ', ' ', 'O', 'X']}
{'player': 'O', 'move': 6, 'board': ['X', 'X', ' ', 'O', ' ', ' ', 'O', 'O', 'X']}
{'player': 'X', 'move': 2, 'board': ['X', 'X', 'X', 'O', ' ', ' ', 'O', 'O', 'X']}
```

Figure 4: An Early Version of the Game Log (Not Secured)

4 RESULTS, DISCUSSION, AND CONCLUSION

In the end, we have proven to have developed a Python program that implements Tic-Tac-Toe in a zk-SNARKs styled manner. Regardless of she wins or lose, Alice always tries to claim she won. It is up to Bob (using the verifier function) to determine if her claims are true or not. The program succeeds at doing such. Figure 5 shows the algorithm verifying her claim whereas Figure 6 refutes said claim.

```
--- CLAIM (Win Detected) ---
  | X | O
O | X |
  | X |
Alice (X) claims she won by securing: (1, 4, 7)

--- PROOF ---
{'commitment': '616e3d69fce457a3b506648f78e7c95f398ea00689be4b5a7b27ba71574e2b1c', 'timestamp': '1775352922.230663'}
{'commitment': '6c40271fbbcf589061f9aae3da71093c7bf5fe724ad63f60b3d05b65ee2a825c', 'timestamp': '1775352922.230728'}
{'commitment': '080eeaa8c72e81d4854603e1b847bd766e4e292d6c7f3bc5c17d92b623aef3af', 'timestamp': '1775352922.2310839'}
{'commitment': '3abbea7d275621cf43216cde39e201df7c1d030a36ad1c2b1f4bd062d3bd5405', 'timestamp': '1775352922.2311144'}
{'commitment': '85588011aed817f7d8a98bb9a54f89f9c3f647b8708b69b4f656e2f5cea7fb69', 'timestamp': '1775352922.2311294'}

--- Verifying Zero-Knowledge Proof ---
Verified: Alice (X) wins.
```

Figure 5: Verifier Function Accepts Alice's Claim

```
--- CLAIM (Win Detected) ---
X | X | O
  | X | X
O | O | O
Alice (X) claims she won by securing: (6, 7, 8)

--- PROOF ---
{'commitment': 'f979d23cc4e4490b74d72e7e8287ea51e225aa016d0429b57150b214c28e2a24', 'timestamp': '1775352889.166496'}
{'commitment': 'aa8f98978233e103e1b7e83ea2f963c72e7e4b1e66f591494471672fde53c076', 'timestamp': '1775352889.166575'}
{'commitment': 'd71a86b4fad1b58b7900819e18fa6ec8b8fc4ae881c28200f002c8bfbfa5394f', 'timestamp': '1775352889.1666005'}
{'commitment': '7065c9abbc28ab5c22a6aeff6483d36d02a832c9142312722d33c10dc991063b6', 'timestamp': '1775352889.166619'}
{'commitment': 'cea87b157adfcc06a95a1d2a6c0917f86c9859dc617c4ebe8a2ef343c30f66e7', 'timestamp': '1775352889.1666338'}
{'commitment': '7a3e0193ea15764f0864913c1a137f4d96cf6239d436017664cebca415026ce8', 'timestamp': '1775352889.1666481'}
{'commitment': '26481e74165fe96276ed0c624d094899fb4673edeb0ccae2a2913ecd0a132a2e', 'timestamp': '1775352889.166662'}
{'commitment': 'a385d36005cc4a06d1904b9ad1b2acb3c5e6c72a5485fdd73462187338c2cc3e', 'timestamp': '1775352889.1666756'}

--- Verifying Zero-Knowledge Proof ---
Verification failed: Alice lost (O wins)
```

Figure 6: Verifier Function Refutes Alice's Claim

4.1 Limitation

While this experiment was successful, we can look into limitations of this experiment and issues future works can analyse. These include:

(1) **Replace randomisation with actual strategic gameplay.** This can be achieved by using developing an AI for both Alice and her rival. These AIs would most likely use reinforcement learning as that is very suitable for working through game strategies.
(2) **Improved Timestamping Systems:** While the current timestamping strategy is somewhat acceptable, it is recommended to instead get trusted timestamps from a trusted authority, such as `RFC1361`.[4]
(3) **Implementation on More Complex Games:** Tic-Tac-Toe in respect to complexity is a mid-tier game. There are simpler games such as *Matching Pennies* or *Rock-Paper-Scissors* as well as more complex games such as *Chess* and/or card games. One can experiment with analysing these latter games under a zk-SNARKs approach.
(4) **A more Fine-tuned zk-SNARKs Approach:** This implementation of zk-SNARKs was simple as it aimed to minimise external libraries. However, with more resources and finer calculations, a truer zk-SNARKs system can be perfected.
(5) **Real-World Applicable Problem:** Usually, the consequence of lying about winning a game is insignificant with regards to the impact on society. However, there are better applications where zero-knowledge proofs can have a greater impact on society.

4.2 Conclusion

We have developed a Python program which can execute a gameplay of Tic-Tac-Toe and certify results without actually knowing about intermediary steps. This was done using zk-SNARKs, and had cryptographic features such as SHA-256 hashing and timestamps.

REFERENCES

[1] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. 2019. Transparent SNARKs from DARK Compilers. Cryptology ePrint Archive, Paper 2019/1229. https://eprint.iacr.org/2019/1229
[2] Consensys. 2017. Introduction to zk-SNARKs. https://consensys.io/blog/introduction-to-zk-snarks Retrieved from Consensys.
[3] Gate.io. 2023. Mina Protocol Explained: A Guide to the Lightweight Blockchain. https://www.gate.com/learn/articles/mina-protocol-explained/863 Accessed: 2026-04-03.
[4] David L. Mills. 1992. Simple Network Time Protocol (SNTP). https://www.rfc-editor.org/rfc/rfc1361.html RFC 1361.
[5] Anca Nitulescu. 2020. zk-SNARKs: A Gentle Introduction. https://www.di.ens.fr/~nitulesc/files/Survey-SNARKs.pdf Technical report, École Normale Supérieure.
[6] H. Oguro. 2024. What is Zero-Knowledge Proof - a hot technology bringing trustworthiness to Web3 privacy? https://www.nttdata.com/global/en/insights/focus/2024/what-is-zero-knowledge-proof Retrieved from NTT Data.
[7] Ali Rahimi and Mohammad Ali Maddah-Ali. 2021. Multi-Party Proof Generation in QAP-based zk-SNARKs. arXiv:2103.01344 [cs.CR] https://arxiv.org/abs/2103.01344
[8] Ashfi Shah. 2026. Tic-Tac-Toe as a Zero-Knowledge Proof Problem. https://github.com/Ashfi-Hasnain/Tic-Tac-Toe-as-a-Zero-Knowledge-Proof-Problem Accessed: 2026-04-04.
[9] D. Wong. 2022. How to Write a Tic-Tac-Toe Zkapp. https://www.cryptologie.net/posts/how-to-write-a-tic-tac-toe-zkapp/ Retrieved from Cryptologie.
[10] ZK Jargon. n.d.. Polynomial Interactive Oracle Proof (PIOP). https://zkjargon.github.io/definitions/polynomial_interactive_oracle_proof.html Accessed: 2026-04-02.

Written 1 April 2026; Submitted 5 April 2026